LABORATORIO AA. 2025/2026

Computazione Quantistica INF0427

Studio sperimentale dell'algoritmo di Grover

Pierluigi Boscaglia

Matricola: 983553

1 Introduzione

In questo progetto studio l'algoritmo di Grover (paper originale), che permette di cercare una soluzione in uno spazio non strutturato mediante l'amplificazione delle ampiezze. L'obiettivo degli esperimenti è capire come cambia la probabilità di successo al variare del numero di iterazioni e quanto il comportamento ideale si discosta da quello osservato in presenza di rumore.

Il percorso parte dalla costruzione visuale dei circuiti con Quantum Programming Studio e IBM Quantum Composer. Prosegue con piccoli esercizi in Python usando Cirq di Google Quantum AI, per seguire passo dopo passo l'evoluzione delle ampiezze e confrontare le probabilità ideali con le frequenze ottenute dalle misure simulate. Infine utilizzo Qiskit per approfondire le simulazioni ideali e con rumore e svolgere prove su hardware reale tramite IBM Quantum Platform.

2 Primi esperimenti con strumenti visuali

Per iniziare ho costruito due piccoli circuiti attraverso strumenti di composizione visuale: il primo con Quantum Programming Studio, il secondo con IBM Quantum Composer. Ho inserito le porte direttamente sulle linee dei qubit, seguendo da sinistra a destra le operazioni di preparazione, oracolo, diffusore e misura. Questo approccio mi ha permesso di ragionare sulle singole operazioni prima di passare alla costruzione dei circuiti in Python.

Entrambi gli esempi utilizzano due qubit di ricerca, quindi quattro stati possibili. Le porte Hadamard iniziali preparano la sovrapposizione uniforme:

$$|s\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

Inizialmente, ciascuno dei quattro stati ha probabilità 1/4 di essere misurato. L'oracolo inverte il segno dell'ampiezza dello stato cercato, senza modificarne la probabilità; il diffusore sfrutta questa differenza di fase per amplificarne l'ampiezza. In entrambi i circuiti ho applicato una sola iterazione di Grover, composta da oracolo e diffusore. Nel seguito rappresento gli stati nell'ordine $|q_1 q_0\rangle$: il qubit q_0, disegnato in alto nel circuito, corrisponde al bit più a destra nella stringa. Mantengo questa convenzione in tutta la relazione per rendere univoca la lettura degli stati.

2.1 Quantum Programming Studio: circuito senza ancilla

Nel primo circuito ho utilizzato soltanto i due qubit di ricerca. L'oracolo è costruito con porte X, Hadamard e una CNOT. Una Hadamard sul target prima e dopo la CNOT permette di ottenere una porta controlled-Z:

$$H_{q_1}\,\mathrm{CNOT}_{q_0\to q_1}\,H_{q_1} = \mathrm{CZ},$$

dove q_0 è il controllo della CNOT e q_1 il suo target. La CZ lascia invariati gli stati della base, tranne $|11\rangle$, di cui inverte il segno dell'ampiezza. Per marcare invece $|10\rangle$, applico una porta X su q_0 prima e dopo questo blocco. La prima X trasforma temporaneamente $|10\rangle$ in $|11\rangle$; la CZ introduce il segno meno e la seconda X ripristina lo stato iniziale:

$$|10\rangle \xrightarrow{X_{q_0}} |11\rangle \xrightarrow{\mathrm{CZ}} -|11\rangle \xrightarrow{X_{q_0}} -|10\rangle.$$

Gli altri stati vengono ripristinati senza cambiamenti di segno, perché la prima X non li porta in $|11\rangle$.

> L'effetto complessivo dell'oracolo è quindi **invertire soltanto il segno dell'ampiezza** della soluzione $|10\rangle$, **senza modificarne i bit o la probabilità di misura**.

Dopo l'oracolo, il diffusore applica H e X a entrambi i qubit, poi una seconda CZ, ancora realizzata con H–CNOT–H sul target, e infine annulla le X e applica le Hadamard finali. Il qubit target delle CNOT fa parte del registro di ricerca: non viene aggiunto un qubit ausiliario.

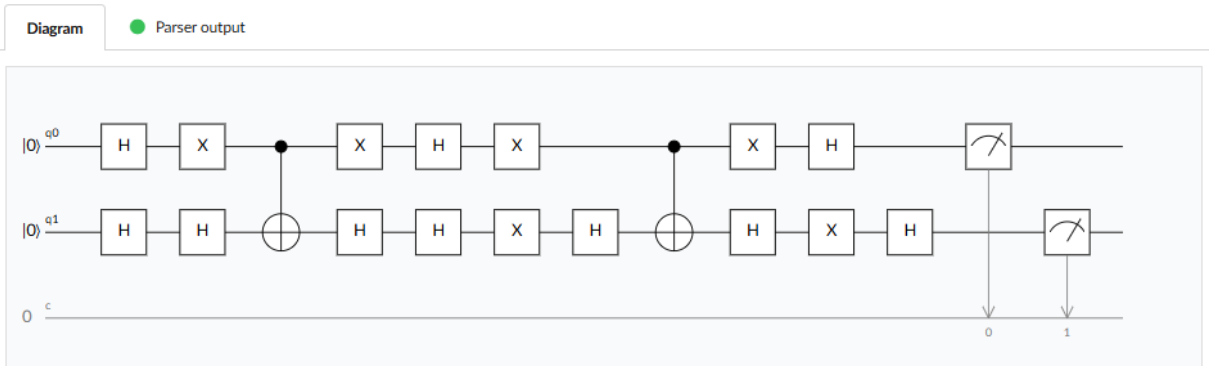


Figura 1: Circuito realizzato in Quantum Programming Studio con due qubit di ricerca, senza ancilla.



Figura 2: Risultato visualizzato in Quantum Programming Studio: $q_0 = 0$ e $q_1 = 1$, corrispondenti allo stato $|10\rangle$ nella convenzione adottata.

2.2 Quantum Composer: circuito con ancilla

Nel secondo circuito ho mantenuto due qubit di ricerca e aggiunto un terzo qubit, indicato con `anc[0]` nello screenshot. Questo qubit non rappresenta un ulteriore bit della soluzione: lo spazio di ricerca rimane di quattro stati. Le porte X e H preparano l'ancilla nello stato

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

L'oracolo è una Toffoli con i due qubit di ricerca come controlli e l'ancilla come target. Poiché $X|-\rangle = -|-\rangle$, quando entrambi i controlli valgono uno si introduce un segno meno nell'ampiezza corrispondente. Nel *phase kickback* l'ancilla permette di modificare la fase dello stato $|11\rangle$, pur rimanendo nello stato $|-\rangle$.

Per costruire il diffusore applico H e X sui qubit di ricerca, riutilizzo la Toffoli con la stessa ancilla, quindi applico nuovamente X e H. Alla fine misuro soltanto i due qubit di ricerca. In Figura 3, l'ancilla è la linea `anc[0]`.

Perché uso una porta X sull'ancilla?
Nello schema mostrato a lezione, il cambio di segno viene ottenuto con una porta Z controllata, applicata direttamente ai qubit di ricerca. Nel mio circuito ottengo lo stesso effetto con una Toffoli e un qubit aggiuntivo: l'ancilla. Per far funzionare questo meccanismo, preparo l'ancilla in $|-\rangle$ applicando prima X e poi H. Quando i due controlli valgono uno, la Toffoli applica una X all'ancilla: poiché $X|-\rangle = -|-\rangle$, cambia il segno dell'ampiezza corrispondente nel registro di ricerca. Le porte H e X sui qubit di ricerca rimangono necessarie per completare il diffusore. I due modi di costruirlo producono le stesse probabilità di misura

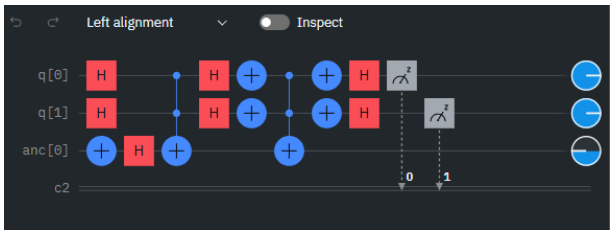


Figura 3: Circuito realizzato in IBM Quantum Composer: due qubit di ricerca e un'ancilla per il phase kickback.

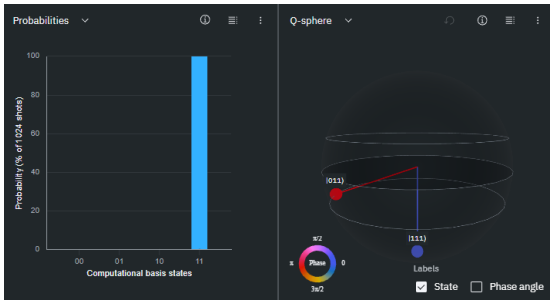


Figura 4: Risultato in Quantum Composer: il registro di ricerca viene misurato nello stato $|11\rangle$. La Q-sphere include anche l'ancilla.

2.3 Confronto e funzionamento dei due circuiti

Le due costruzioni realizzano la stessa idea di Grover con risorse diverse. Il primo circuito modifica direttamente le fasi del registro attraverso una CZ (decomposta come visto); il secondo ottiene la modifica di fase usando un'ancilla. Questa differenza dipende dalla costruzione scelta, non da un obbligo imposto dai due strumenti grafici. Inoltre i target sono diversi: $|10\rangle$ nel primo circuito e $|11\rangle$ nel secondo.

Dopo l'oracolo l'ampiezza della soluzione vale $-1/2$ e le altre valgono $1/2$. La loro media è $1/4$: il diffusore standard trasforma ogni ampiezza a in $2\overline{a} - a$, portando quella della soluzione a 1 e le altre a 0. Le sequenze mostrate realizzano il diffusore a meno di un segno globale, che non modifica le probabilità di misura.

3 Esercizio svolto con Cirq di Google Quantum AI

Dopo i circuiti costruiti con gli strumenti visuali, ho realizzato un piccolo esperimento in Python con Cirq di Google Quantum AI. Come Qiskit, Cirq è una libreria Python; una sua particolarità è organizzare esplicitamente il circuito in momenti, raggruppando le operazioni su qubit distinti: una struttura comoda per seguire la simulazione passo dopo passo. Il codice viene eseguito su un simulatore ideale locale, senza collegarsi a un computer quantistico. Ho scelto tre qubit di ricerca, quindi otto stati possibili, e una sola soluzione da trovare: $|101\rangle$, nell'ordine $|q_2 q_1 q_0\rangle$.

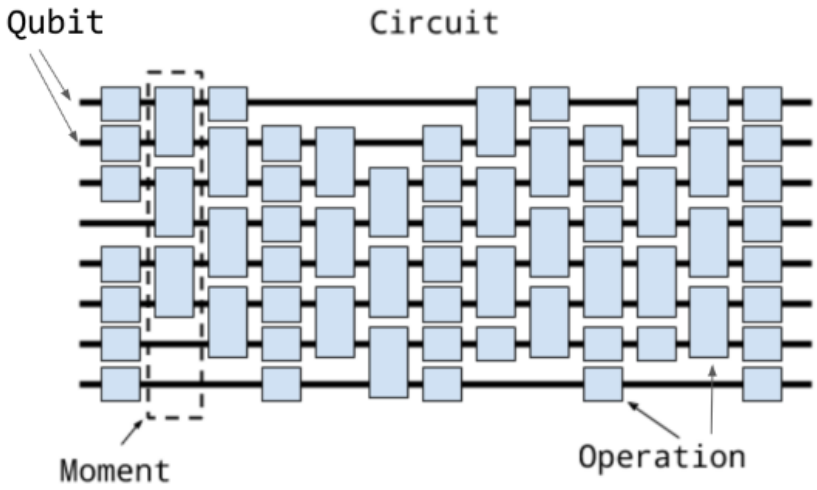


Figura 5: Organizzazione di un circuito in Cirq: qubit, operazioni e momenti.

Costruzione del circuito. Ho mantenuto la costruzione con ancilla usata in Composer: il circuito contiene quindi quattro qubit totali, ma solo tre codificano la soluzione. Per selezionare $|101\rangle$, invece di $|111\rangle$, inserisco una X sul qubit centrale q_1 prima e dopo questa porta. Il phase kickback cambia così il segno della sola ampiezza cercata. Il diffusore riutilizza la stessa ancilla, con le porte H e X sui qubit di ricerca prima del blocco controllato e X e H dopo.

Osservazione dei passaggi. Nel notebook stampo il circuito durante la costruzione e seguo la simulazione dopo ogni *momento*, cioè un gruppo di operazioni su qubit distinti. Per ogni stato di ricerca mostro le due ampiezze associate ai valori zero e uno dell'ancilla. La probabilità del registro di ricerca si ottiene sommando i moduli al quadrato di queste due ampiezze. Queste informazioni sono disponibili nel simulatore: non sono misure intermedie inserite nel circuito. Le stampe permettono di osservare che l'oracolo completo cambia il segno delle ampiezze senza cambiare le probabilità, mentre il diffusore aumenta quella della soluzione. Verifico inoltre che, dopo la preparazione, l'ancilla rimanga nello stato $|-\rangle$.

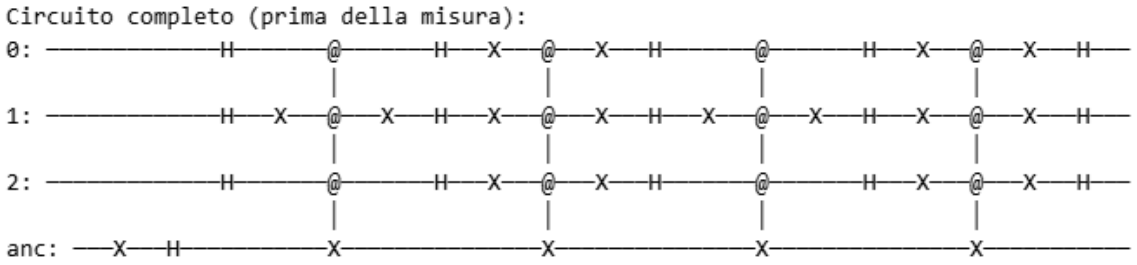


Figura 6: Circuito di Grover costruito con Cirq prima della misura: tre qubit di ricerca, un'ancilla e due iterazioni.

```
Momento 3: Sovrapposizione iniziale
Ricerca   Amp(anc=0)   Amp(anc=1)   Prob. ricerca
000       +0.25000      -0.25000        12.500%
001       +0.25000      -0.25000        12.500%
010       +0.25000      -0.25000        12.500%
011       +0.25000      -0.25000        12.500%
100       +0.25000      -0.25000        12.500%
101       +0.25000      -0.25000        12.500%
110       +0.25000      -0.25000        12.500%
111       +0.25000      -0.25000        12.500%
>>> Fine Sovrapposizione: P(101) = 12.50000%
```

Momento 3

```
Momento 6: Oracolo: ripristino del bit centrale
Ricerca   Amp(anc=0)   Amp(anc=1)   Prob. ricerca
000       +0.25000      -0.25000        12.500%
001       +0.25000      -0.25000        12.500%
010       +0.25000      -0.25000        12.500%
011       +0.25000      -0.25000        12.500%
100       +0.25000      -0.25000        12.500%
101       -0.25000      +0.25000        12.500%
110       +0.25000      -0.25000        12.500%
111       +0.25000      -0.25000        12.500%
>>> Fine Oracolo 1: P(101) = 12.50000%
```

Momento 6

```
Momento 11: Diffusore: H
Ricerca   Amp(anc=0)   Amp(anc=1)   Prob. ricerca
000       -0.12500      +0.12500         3.125%
001       -0.12500      +0.12500         3.125%
010       -0.12500      +0.12500         3.125%
011       -0.12500      +0.12500         3.125%
100       -0.12500      +0.12500         3.125%
101       -0.62500      +0.62500        78.125%
110       -0.12500      +0.12500         3.125%
111       -0.12500      +0.12500         3.125%
>>> Fine Diffusore 1: P(101) = 78.12500%
```

Momento 11

```
Momento 19: Diffusore: H
Ricerca   Amp(anc=0)   Amp(anc=1)   Prob. ricerca
000       -0.06250      +0.06250         0.781%
001       -0.06250      +0.06250         0.781%
010       -0.06250      +0.06250         0.781%
011       -0.06250      +0.06250         0.781%
100       -0.06250      +0.06250         0.781%
101       +0.68750      -0.68750        94.531%
110       -0.06250      +0.06250         0.781%
111       -0.06250      +0.06250         0.781%
>>> Fine Diffusore 2: P(101) = 94.53125%
```

Momento 19

Figura 7: Stampe della simulazione in quattro momenti del circuito di Grover.

Risultati. Ho applicato due iterazioni di Grover. I grafici del notebook mostrano come cambia la distribuzione sugli otto stati; per la soluzione si ottiene:

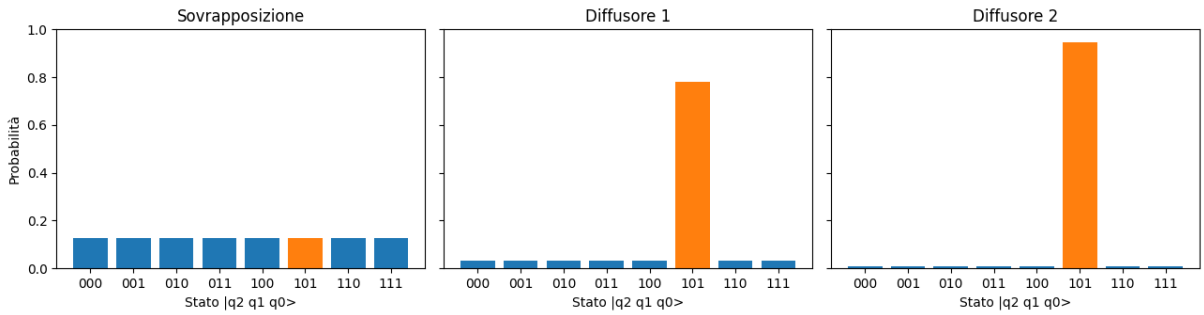


Figura 8: Distribuzione delle probabilità nella simulazione con Cirq: sovrapposizione iniziale e risultati dopo il primo e il secondo diffusore. Lo stato cercato $|101\rangle$ è evidenziato in arancione.

Il risultato coincide con la previsione teorica $P_k = \sin^2((2k+1)\theta)$, con $\theta = \arcsin(1/\sqrt{8})$. Infine ho misurato soltanto i tre qubit di ricerca per 1024 ripetizioni: nell'esecuzione salvata nel notebook, 101 compare 964 volte, con una frequenza di circa 94,14%.

Stato	Conteggi	Frequenza	Probabilità ideale
000	4	0.39%	0.78%
001	9	0.88%	0.78%
010	9	0.88%	0.78%
011	13	1.27%	0.78%
100	10	0.98%	0.78%
101	964	94.14%	94.53%
110	11	1.07%	0.78%
111	4	0.39%	0.78%

Figura 9: Risultati di 1024 shot nella simulazione con Cirq.

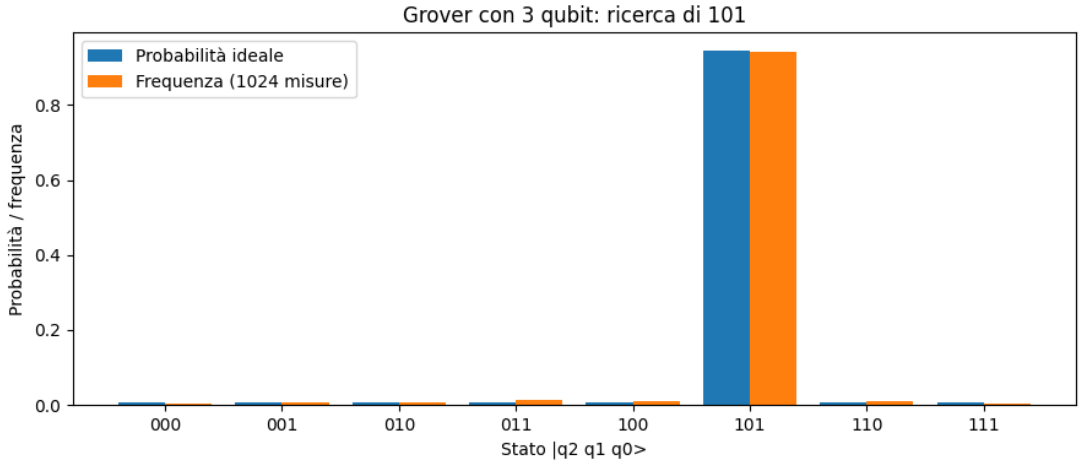


Figura 10: Confronto grafico tra le probabilità.

4 Studio Sperimentale con Qiskit di IBM Quantum e hardware reale

Dopo i primi esercizi, ho approfondito lo studio di Grover nel notebook `Grover.ipynb`, utilizzando Qiskit. L'obiettivo è verificare il comportamento previsto dalla teoria e capire come cambiano i risultati quando si passa dalla simulazione ideale a condizioni più vicine a quelle di un dispositivo reale.

Ho organizzato gli esperimenti in tre modalità:

- simulazione ideale,
- simulazione con rumore ricavato dai profili dei dispositivi IBM (*fake backend*),
- esecuzione su hardware quantistico IBM.

Nella simulazione ideale confronto le probabilità ricavate dal vettore di stato con le frequenze osservate nelle misure, per mettere in evidenza le fluttuazioni dovute al numero finito di ripetizioni. Studio inoltre il caso in cui il numero di soluzioni non sia noto, scegliendo casualmente quante iterazioni eseguire e verifico se il risultato è una soluzione valida. Passo poi alle simulazioni con rumore e alle prove su hardware reale, per osservare come gli errori delle porte e della lettura modifichino la probabilità di successo rispetto al caso ideale.

4.1 Ambiente di lavoro e strumenti utilizzati

- **Python, ambiente virtuale e notebook in Visual Studio Code.** Il codice viene eseguito nell'ambiente locale `.venv`, con un kernel Python per i notebook Jupyter. Questo formato permette di affiancare codice, spiegazioni, grafici e risultati. Le dipendenze del progetto sono elencate in `requirements.txt`.
  notebook Jupyter in VS Code.

- **Qiskit: costruzione e verifica dei circuiti.** Uso `QuantumCircuit` per costruire l'oracolo, il diffusore e il circuito completo, inserendo esplicitamente le porte e le misure. Le classi `Operator` e `Statevector` permettono di controllare gli operatori e calcolare ampiezze e probabilità ideali.
  documentazione Qiskit.

- **Transpiler di Qiskit.** Uso `transpile` e il generatore di pass manager `generate_preset_pass_manager` per adattare i circuiti alle porte e ai collegamenti disponibili sul backend scelto. Questo passaggio è necessario soprattutto prima dell'esecuzione sull'hardware reale.
  transpiler di Qiskit.

- **Qiskit Aer.** La classe `AerSimulator` esegue localmente i circuiti con misure ripetute, sia nel caso ideale sia in presenza di rumore. Il metodo `from_backend` configura il simulatore a partire dalle caratteristiche di un backend.
  AerSimulator.

- **Fake backend IBM.** Uso `FakeManilaV2`, `FakeNairobiV2` e `FakeMumbaiV2`, importati da `qiskit_ibm_runtime.fake_provider`, per impostare le simulazioni con rumore. Questi profili descrivono caratteristiche ed errori di dispositivi IBM; le prove rimangono simulazioni locali e non riproducono necessariamente le condizioni attuali delle QPU.
  fake provider IBM.

- **IBM Quantum Platform e Qiskit IBM Runtime.** Accedo all'hardware tramite il mio account IBM. `QiskitRuntimeService` gestisce la connessione, la scelta del backend e il recupero dei job; `SamplerV2` invia i circuiti e restituisce i risultati delle misure. Salvo i risultati per poterli analizzare anche senza ripetere l'esecuzione sulla QPU.
  API di Qiskit IBM Runtime.

4.2 Descrizione e implementazione dell'algoritmo

L'algoritmo di Grover cerca una soluzione tra $N = 2^n$ stati, rappresentati da n qubit. Nel notebook indico con W l'insieme delle soluzioni e con M il loro numero. Per costruire e verificare gli esperimenti scelgo esplicitamente le stringhe da marcare: non carico un database classico, ma realizzo un oracolo che riconosce gli stati scelti. Le stringhe seguono la convenzione $|q_{n-1} \dots q_0\rangle$.

Funzionamento. Il circuito parte dallo stato $|0 \dots 0\rangle$. Una porta Hadamard su ciascun qubit prepara la sovrapposizione uniforme

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

Ogni iterazione applica poi due operazioni:

1. **Oracolo.** Cambia il segno dell'ampiezza degli stati appartenenti a W, lasciando invariati gli altri:
$$O_W|x\rangle = \begin{cases} -|x\rangle & \text{se } x \in W, \\ |x\rangle & \text{altrimenti.} \end{cases}$$
Questo passaggio modifica le fasi, ma non le probabilità di misura.

2. **Diffusore.** Applica la riflessione $D = 2|s\rangle\langle s| - I$, che trasforma ogni ampiezza a_x in $2\overline{a} - a_x$, dove $\overline{a}$ è la media delle ampiezze. Insieme all'oracolo, permette di amplificare le soluzioni.

Dopo k iterazioni lo stato è $(DO_W)^k|s\rangle$ e la misura restituisce una stringa candidata. Nel caso ideale la probabilità complessiva di ottenere una soluzione è

$$P(k) = \sin^2((2k+1)\theta), \qquad \theta = \arcsin\sqrt{\frac{M}{N}}.$$

Codice sorgente e scelte implementative. Il codice è scritto in Python con Qiskit e si trova nelle sezioni 5–8 di `Grover.ipynb`. Le funzioni principali sono:

- `build_phase_oracle`: per ogni soluzione applica X ai qubit corrispondenti ai bit zero, una Z multicontrollata e infine annulla le X. Il cambio di fase su $|11\ldots1\rangle$ viene così trasferito alla stringa desiderata.
- `build_diffuser`: usa `grover_operator` con un oracolo identità per ottenere soltanto il diffusore D. La fase globale necessaria al confronto delle ampiezze con la teoria è già inclusa da Qiskit e non viene aggiunta nuovamente.
- `build_grover_circuit`: prepara la sovrapposizione, compone oracolo e diffusore per il numero richiesto di iterazioni e aggiunge, se richieste, le misure finali.

La funzione di supporto `apply_multi_controlled_z` usa una Z per un qubit e `MCMTGate(ZGate(), n - 1, 1)` per due o più qubit. A differenza dell'esercizio con Cirq, qui il circuito logico non aggiunge un'ancilla dedicata: il target della porta appartiene al registro di ricerca. La compilazione per il backend viene svolta successivamente. Per le verifiche con `Statevector` costruisco il circuito senza misure.

Materiale di riferimento. La costruzione segue l'algoritmo descritto nel materiale teorico del corso. Nel notebook sono inoltre richiamate le spiegazioni di IBM Quantum sull' interpretazione geometrica e sulla scelta delle iterazioni. Per il codice di oracolo e diffusore ho adottato una costruzione molto vicina agli esempi della documentazione IBM, adattandola alle funzioni e ai parametri utilizzati negli esperimenti del notebook:

- **Oracolo:** il tutorial IBM sull'algoritmo di Grover presenta la funzione `grover_oracle`, che applica X ai bit zero della soluzione, una Z multicontrollata e infine le stesse X. La funzione `build_phase_oracle` è un adattamento di questo codice: mantiene l'inversione della stringa per rispettare l'ordine dei qubit e usa la stessa porta `MCMTGate` con `ZGate`.
- **Diffusore:** la documentazione Qiskit di `grover_operator` viene utilizzata direttamente nella funzione `build_diffuser`. Poiché `grover_operator` costruisce il prodotto DO, passo come oracolo un circuito vuoto, che realizza l'identità: ottengo così soltanto D.

Il codice utilizza quindi esempi e strumenti forniti da IBM, evitando cosi costruzioni sviluppate da zero.